

Project Overview- Contract Reviewer

A multi-agent system that reads any contract PDF, checks every clause against legal references, flags risks, and returns a structured report with plain-English summaries and negotiation priorities.

1. Tech Stack

Category	Technology
Orchestration	LangGraph
LLM + RAG Chains	LangChain
Tracing & Observability	LangFuse
LLM Inference	Azure OpenAI
Vector Search	Azure AI Search
PDF Parsing	Azure DI
Structured Output	Pydantic v2
API Layer	FastAPI
Frontend Prototype	Streamlit
State Persistence	redis
User Data & MetaData	SupaBase
PDF	Azure Blob

2. Agent Design

Agent 1,6 runs initially while Agents 2–5 run in **parallel using LangGraph's `Send()` API**.

Each agent has an isolated responsibility and receives its own RAG context slice pre-fetched by the supervisor node.

2.1 Clause Extractor

Execution: Sequential

Model: GPT-4.1 Mini

Estimated Cost: ~\$0.0014 per contract

Responsibilities

- Parse PDF into labeled clause units
- Segment clauses using heading hierarchy and section boundaries

Output Fields

- Clause type
 - Raw text
 - Section reference
-

2.2 Risk Scorer

Execution: Parallel

Model: GPT-4.1

Estimated Cost: ~\$0.19 per contract

Responsibilities

- Compare clauses against standard legal positions from RAG
 - Score deviations from accepted standards
 - Assign RiskLevel
 - Generate negotiation suggestions
-

2.3 Obligation Finder

Execution: Parallel

Model: GPT-4.1 Mini

Estimated Cost: ~\$0.016 per contract

Responsibilities

- Extract:
 - Deadlines
 - Payment terms
 - Notice periods
 - Restrictions
 - Compare obligations against historical contract patterns
-

2.4 Red Flag Detector

Execution: Parallel

Model: GPT-4.1 Mini

Estimated Cost: ~\$0.013 per contract

Responsibilities

- Match clauses against curated red-flag patterns
 - Explain why language is risky
 - Suggest safer alternatives
-

2.5 Plain English Writer

Execution: Parallel

Model: GPT-4.1 Nano

Estimated Cost: ~\$0.0018 per contract

Responsibilities

- Rewrite legal clauses into simple language
- Produce concise explanations understandable by non-lawyers

Reason

No reasoning or retrieval required. Pure language transformation task.

2.6 Report Assembler

Execution: Sequential

Model: GPT-4.1 Mini

Estimated Cost: ~\$0.003 per contract

Responsibilities

- Compute overall verdict
 - Rank negotiation priorities
 - Detect missing clauses
 - Assemble final report object
-

3. LangGraph Design

The system will be implemented as a **LangGraph** :

- Typed shared state
 - Parallel execution
 - Persistent checkpointing
-

Key Architectural Patterns

Send() Parallel Fan-Out

The supervisor dispatches agents 2–5 simultaneously using `send()`

Each agent receives:

- Shared clause list
 - Agent-specific RAG context
-

List Reducers for Safe Concurrent Writes

Parallel agents write safely using reducers which prevents state overwrites during concurrent execution.

Redis Checkpointing

State is persisted after every node execution.

Benefits

- Crash recovery
 - Resume from checkpoints
 - Enables future human-in-the-loop workflows
-

LangFuse Tracing

Tracks:

- Agent execution
 - Retrieval quality
 - Token usage
 - Latency breakdown
 - State transitions
-

4. RAG Design

The project will use **3 specialized RAG indexes** with hybrid retrieval:

- BM25 keyword search
- Dense vector search

Hybrid retrieval is essential because legal terminology often depends on exact wording.

4.1 Legal Standards Library

Contents

- Contract union guides
- Consumer protection documents
- Standard templates
- Jurisdiction-specific contractor rights

Chunking Strategy

Chunked by clause type.

Used By

- Risk Scorer

Retrieval

- Hybrid search
 - Metadata filtering by `clause_type`
-

4.2 Sample Contracts Index

Contents

- Open-source templates
- Anonymized contracts
- CommonPaper examples

Metadata

- Contract type
- Clause type
- Outcome:
 - Signed
 - Rejected

- Negotiated

Used By

- Obligation Finder

Retrieval

- Semantic search
 - Filtered by `contract_type`
-

4.3 Red Flag Library

Contents

Collection containing:

- Dangerous clause patterns
- Explanation of risks
- Suggested safer replacements

Characteristics

- Precision-focused
- Human-curated
- Not scraped automatically

Used By

- Red Flag Detector

Retrieval

- Keyword-first hybrid retrieval
 - Top-3 retrieval strategy
-

5. Structured Output Models

All agents return validated Pydantic models

6. Model Selection & Costing

The system will use **mixed-model routing** where each agent receives the minimum model capability required.

Cost Comparison

Configuration	Estimated Cost
Mixed Routing (Recommended)	~\$0.24
GPT-4.1 Everywhere	~\$0.48
GPT-4o Everywhere	~\$0.60

Per-Agent Cost Breakdown

Agent	Model	Why This Model	Cost
Clause Extractor	GPT-4.1 Mini	Structured extraction only	\$0.0014
Risk Scorer	GPT-4.1	Advanced legal reasoning	\$0.19
Obligation Finder	GPT-4.1 Mini	Pattern recognition	\$0.016
Red Flag Detector	GPT-4.1 Mini	RAG-driven classification	\$0.013
Plain English Writer	GPT-4.1 Nano	Simple language transformation	\$0.0018
Report Assembler	GPT-4.1 Mini	Aggregation and verdict generation	\$0.003

7. Edge Cases & Failure Handling

Scanned PDFs

Problem

No extractable text.

Solution

Fallback OCR using:

- Azure AI Document Intelligence
-

Clauses Split Across Pages

Solution

Segment using document structure rather than page boundaries.

Missing Standard Clauses

Solution

Detect absent clause types by comparing against expected contract structure.

Contradictory Clauses

Solution

Cross-check clause types and detect conflicting definitions.

Jurisdiction Variance

Solution

RAG retrieval filtered using jurisdiction metadata.

Handwritten Amendments

Status

Not supported automatically.

Behavior

Flag for manual review.

Password-Protected PDFs

Handling

Rejected during upload validation with clear error response.

Structured Output Validation Failure

Strategy

- Retry generation up to 2 times
 - Append validation error to retry prompt
 - Preserve failed state
-

Contract Reviewer — System Architecture

